CELESTIAL MECHANICS × AI · 2026

From the next token to the next theorem



Bowen Liu

Bowen Liu 刘博文

Periodic N-body solutions, elliptic relative equilibria and linear stability.

2013–2018 **Nankai University** · Chern Institute · Math PhD

2018–2020 **University of Augsburg** · Postdoc

2020–2022 **Shanghai Jiao Tong** · Wu Wen-Tsun Asst. Prof.

2022–2024 **Huawei** · Principal Engineer

2024–now **US e-commerce** · Applied Researcher

CELESTIAL MECHANICS RESEARCH

Periodic N-body: existence & stability

Restricted four-body elliptic equilibria

Linear instability of elliptic rhombi

Homographic Lennard–Jones solutions

> Symplectic indices · Variational methods

After training: four limits, three system capabilities

The model predicts; state, computation and reliability live outside it.

01 · MEMORY & CONTEXT

No persistent state, limited window

- Parameters fixed at inference; knowledge in weights
- A “conversation” resends history
- 4k → 1M, but long inputs still lose the middle

02 · COMPUTATION IS RECALL

Recall and pattern completion

- Arithmetic comes from training patterns, not an algorithm
- Times table: recall, not algorithm
- Errors compound over long tasks

03 · PROBABILISTIC OUTPUT

Same input, different output

- Output is sampled, not a deterministic function
- Errors can sound confident and plausible
- Long runs need stopping and rollback

> Four limits, three system capabilities: **state, computation, reliability**.

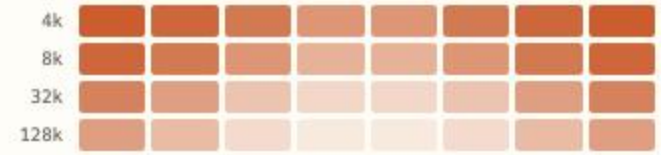
Context and memory: a finite window, used well

The window is a budget, not a warehouse: place the right things, then decide what stays.

01 · FINITE WINDOW

4k → 1M, still lossy

Needle in a haystack: the middle is diluted first.



Capacity ≠ effective use.

02 · PLACEMENT

Workbench, not warehouse

- system · rules
- history · recent turns
- retrieval · relevant docs
- tools · returned results
- current · this question

Too long→retrieve; too scattered→structure; overflow→summarize or sub-agents.

03 · RETENTION

Signal, not logs

Signal
tasks · constraints · decisions · facts

Noise
chat · repetition · stale · raw logs

WRITE
decisions and facts only

RETRIEVE
for this task

COMPRESS
summarize, merge

FORGET
delete stale

> Capacity ≠ effective use; Context decides **what goes in**, Memory decides **what stays**.

Harness: turn probabilistic output into a runnable system

The model proposes; the harness makes the proposal legal, executable, checkable, and stoppable.

A controlled loop

Propose → Validate → Execute → Observe → Verify → Decide

ENGINEERING BOUNDARY

The model only proposes token distributions and tool calls; the security and determinism of the environment are enforced by the **Harness**.

> It does not add intelligence; it makes errors **visible, stoppable, reproducible**.

01 TOOL REGISTRATION & VALIDATION
Check schemas and types; block illegal proposals immediately

02 SANDBOX & PERMISSIONS
What may run; read/write isolation, constrain before execution

03 TIMEOUTS & RETRIES
Prevent infinite loops; enforce timeout limits and retry caps

04 RESULT VERIFICATION
Check correctness via independent assertions; never trust self-claims

05 ROLLBACK & STOPPING
Undo bad changes via snapshots; force-stop on error thresholds

06 LOGS & REPLAY
Record every call and response; ensure full trajectory auditability

> The model provides probability; the harness provides the **boundary**.

Put the model back in the system



01	**LLM**	A probability generator: tokens in, a distribution over the next token out
02	**Prompt**	This call's conditions: system / user / few-shot
03	**Context**	Every token visible to this call; the window is finite
04	**Memory**	State beyond the window: write / retrieve / compress / forget
05	**Agent loop**	Observe → act → observe; state lives outside
06	**Harness**	Tools, sandbox, permissions, verification, logs

> Memory owns state, tool calls compute, harness verifies: together they form an **agent system**.

Theorem-cracking system: LLM as CPU

Not just a mathematics problem; a machine for throughput and stability.

WORKLOAD	**Theorem · proof graph**	what to solve
CPU	**LLM**	propose next
SCHEDULER	**Loop + Harness**	concurrency · retries · checkpoints · stop
VERIFIER	**Lean / tools**	correctness · compute
STATE	**Context + Memory**	graph · logs · budget

01 · NOT ONLY MATHEMATICS

Decomposition, scheduling, state, verification

02 · THROUGHPUT

Concurrency, caching, reclamation, cost

03 · STABILITY

Long runs, recovery, auditability

04 · SCALABLE

Swap problems, not the system

> The model is the CPU; **theorem cracking needs the whole machine**.

Failure probability: long chains amplify 1%

The proof graph is read and written constantly; errors compound along dependencies.

END-TO-END SUCCESS · ILLUSTRATIVE

Per step	100	1,000	10,000
99%	37%	0.0043%	≈0
99.9%	90%	37%	0.0045%
99% + 3 retries	≈100%	99.9%	99%

Assumes independent failures; real errors correlate, and retries are not free.

DETECT ERRORS

Lean, tests, assertions first

RETRY SAFELY

Backoff, caps, no duplicate effects

ROLL BACK STATE

Checkpoints, idempotency, recovery

> The system does not prevent every error; **it makes errors non-fatal**.

Systems engineering: six interfaces

Each layer can work alone; the whole machine still may not be reliable.

01 · STATE CONSISTENCY
Concurrent graph edits
Versions, locks, conflict merge

02 · SCHEDULING & BUDGET
Who runs, how long
Concurrency, preemption, cost

03 · FAILURE & RETRY
No duplicate effects
Idempotency, backoff, caps

04 · VERIFICATION INTERFACE
Natural language ↔ Lean
Semantic gaps and assumptions

05 · OBSERVABILITY
Attribute failures
Logs, replay, audit

06 · HUMAN BOUNDARY
When to stop, who owns
Automate execution, gate judgment

Low-hanging fruit: formal, scorable, cheap to verify · **Real systems problem**: reproducible, scalable, maintainable

> Low-hanging fruit proves direction; **the hard part is the whole machine**.

From the next token to the next theorem

`token` → `probability` → `candidate` → `verification` → **`theorem`**

Probability gives breadth

Verification gives correctness

People set the direction

> The next theorem still begins with the **next token**.